

The `posim` Command Language

Complete Grammar & Notebook Guide

physical_object_simulator

July 21, 2026

Contents

1	What is this document about?	2
2	Lexical structure (what the lexer sees)	2
2.1	Token kinds	3
2.2	Keywords	3
2.3	Magics	4
3	The grammar (what the parser accepts)	4
4	The type system (what values exist)	6
4.1	Special functions	6
4.2	Complex numbers	7
5	Command semantics	8
5.1	NEW — create an object	8
5.2	SET / GET — write and read any field	10
5.3	STEP and RUN — integrate (always via SUNDIALS)	12
5.4	METHOD — choose the integrator	12
5.5	Observables, bookkeeping, help	13
5.6	SCENE — the graphical scene window	13
5.7	COLLIDE and CONTACTS — rigid-body collisions	15
5.8	BOX — the rigid bounding box	15
5.9	User definitions: DEF, LET, FUNCS, SHOW	16
6	The notebook (cells, editing, magics)	18
6.1	Cells	18
6.2	State is cumulative	18
6.3	Magics	19
6.4	Scripts	19
7	JupyterLab in one paragraph	19
8	The stack machine (what runs your line)	19

9	Fifteen worked examples	20
9.1	Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass	20
9.2	Example 2 — a thrown ball vs. the textbook formula	20
9.3	Example 3 — cyclotron motion: expressions as arguments	21
9.4	Example 4 — the tennis-racket theorem (Dzhanibekov effect)	21
9.5	Example 5 — three bodies, then surgery with DEL	22
9.6	Example 6 — magnetic torque (and why ENERGY grows)	22
9.7	Example 7 — fixing a typo in an old cell with %edit	23
9.8	Example 8 — checking the simulator against itself	23
9.9	Example 9 — hand-built tensors and a quaternion orientation	24
9.10	Example 10 — reproducibility: %save , %reset , %load	24
9.11	Example 11 — watching a Kepler orbit live, then running it backward	25
9.12	Example 12 — driving the camera from the notebook	26
9.13	Example 13 — the box of shapes: every body type in a rigid, infinitely massive box	27
9.14	Example 14 — two dumbbells from a user-defined function	29
9.15	Example 15 — a particle in a box, solved inside the language	30
10	Error message tour	32

1 What is this document about?

posim is the front end of the **physical_object_simulator**. Instead of writing Rust code to create objects and run physics, you type short *commands* — one per line — into a *notebook*. Each command line is:

1. chopped into *tokens* by a *lexer* (like the classic Unix tool **lex/flex**);
2. checked against a *grammar* and compiled into a small program by a *parser* (like **yacc/bison**);
3. executed by a *stack machine* — a tiny computer whose only memory is a stack of values — which reads and writes the simulator exclusively through its public *get/set* functions.

You never see steps 1–3; you type a line, press Enter (or shift-enter in JupyterLab), and see the result. This document specifies exactly what you may type.

There are three ways to talk to the same language:

mode	start with	who it is for
notebook REPL	cargo run (or posim)	humans at a terminal
script	posim -script file	batch runs, reproducibility
dynamic notebook machine	posim -notebook file posim -machine	load a notebook, open its scene window, continue live programs, and the JupyterLab kernel

2 Lexical structure (what the lexer sees)

A command line is a sequence of *tokens* separated by optional whitespace. Everything from a **#** to the end of the line is a *comment* and is ignored.

2.1 Token kinds

token	examples	rules
keyword	NEW, set, Run	case-insensitive; list in §2.2
identifier	obj0, position	a letter or <code>_</code> , then letters, digits, <code>_</code>
string	"ball", "dumbell10"	double-quoted, with the escapes <code>\</code> " <code>\\</code> <code>\n</code> ; strings name objects (NEW ... AS, §5.1) and are passed to user functions (§5.9)
number	2, 0.5, .5, 1e-3	64-bit float; scientific notation allowed; a leading <code>-</code> is <i>not</i> part of the number — it is the negation operator
punctuation	[] { } () , . =	brackets build vectors, braces enclose initializers, <code>.</code> builds dotted paths
operators	+ - * /	ordinary arithmetic

If the lexer meets a character it does not know (say `@`), it stops with an error naming the *column* (character position, starting at 1):

```
Err[1]: lexical error at column 10: unexpected character '@'
```

2.2 Keywords

```
NEW SET GET DEL DELETE LIST STEP RUN STEPS METHOD ADAMS BDF SPRK ENERGY COM MOMENTUM
ANGMOM LAPLACE HELP POINT SPHERE CUBOID TORUS DISK CYLINDER BOX RESET
```

and, for the graphical scene window (documented in `scene_info.pdf`; Examples 11–12 show live sessions):

```
SCENE CREATE CLOSE DESTROY TRANSLATE ROTATE ZOOM IN OUT HIDE SHOW REFRESH REDRAW START
STOP PAUSE REVERSE SET_TIME_STEP SETTIMESTEP STATUS EVENTS ALL
```

and, for rigid-body collisions (§5.7):

```
COLLIDE CONTACTS ON OFF
```

and, for named objects, session variables and user functions (§5.1, §5.9):

```
AS LET FUNCS DUMBELL
```

(DESTROY is an alias for CLOSE; SETTIMESTEP for SET_TIME_STEP; the spellings CUBE and DISC are lexer aliases for CUBOID and DISK; FUNCTIONS for FUNCS; and the single-b spelling DUMBELL for DUMBELL. `collisions` is deliberately **not** a keyword so the field path `system.collisions` keeps its own spelling. DEF is not a keyword either: a line starting DEF is a **line form** recognized before the ordinary grammar — see §3 and §5.9.)

Keywords are reserved *at the start of paths*, but **field names may reuse keyword spellings** — `obj0.momentum` and `system.method` work even though MOMENTUM and METHOD are keywords, because after a dot (or inside NEW { ... }) the parser accepts either an identifier or a keyword as a field name. The same holds for the scene keywords: making SHOW or IN reserved does not stop you from ever using those spellings as field names after a ..

2.3 Magics

A line whose first character is % is a *notebook magic* (§6.3) and is handled by the notebook itself; it never reaches the lexer.

3 The grammar (what the parser accepts)

The grammar in EBNF (Extended Backus–Naur Form: [x] means optional, {x} means zero-or-more repetitions, | separates alternatives):

```

line      := command | magic ;

command   := "NEW" shape [ "AS" (IDENT | STRING) ]
              [ "{" init { "," init } "}" ]
              (* AS registers a user
                name for the object *)

          | "SET" path "=" expr
          | "GET" path
          | "DEL" NUMBER
          | "LIST"
          | "STEP" expr          (* advance by dt      *)
          | "RUN" expr [ "STEPS" NUMBER ]      (* advance by t, n outs *)
          | "METHOD" ( "ADAMS" | "BDF" | "SPRK" IDENT [ NUMBER ] )
          | "ENERGY" | "COM" | "MOMENTUM" | "ANGMOM"
          | "LAPLACE" NUMBER
          | "RESET" | "HELP"
          | "SCENE" scenecmd      (* graphical scene      *)
          | "COLLIDE" [ "ON" | "OFF" ]      (* bare: report status *)
          | "CONTACTS"            (* list last contacts   *)
          | "LET" IDENT "=" expr      (* session variable     *)
          | "FUNCS"                (* list user functions  *)
          | "SHOW" IDENT            (* print a function     *)
          | "BOX" [ "OFF" | expr ]      (* rigid bounding box:
                                          expr = inner side
                                          length; bare = status;
                                          OFF removes it      *)
          | expr ;                  (* bare expression      *)
scenecmd  := "CREATE" [ NUMBER ]      (* open window [port]   *)
          | "CLOSE"                  (* aka DESTROY          *)
          | "TRANSLATE" term term [ term ]      (* camera dx dy [dz]   *)
          | "ROTATE" term term          (* camera dyaw dpitch  *)
          | "ZOOM" ( "IN" | "OUT" | term )      (* factor > 1 zooms in *)
          | "HIDE" [ NUMBER | "ALL" ]      (* default: ALL         *)
          | "SHOW" [ NUMBER | "ALL" ]
          | "REFRESH"                  (* re-sync from state  *)
          | "REDRAW"                   (* re-send full scene  *)
          | "START" | "STOP" | "PAUSE" | "REVERSE"
          | "RESET"                    (* re-initialize: all
                                          values and the time
                                          return to initial;
                                          START re-starts    *)
          | "SET_TIME_STEP" term        (* args are term-level:
                                          -5 is negative five;

```

```

                                parenthesize sums *)
    | "STATUS" | "EVENTS" ;
shape := "POINT" | "SPHERE" | "CUBOID" | "TORUS" | "DISK" | "CYLINDER"
      | "DUMBBELL" ;
                                (* two solid spheres +
                                a rigid rod, one
                                rigid body *)

(* User-defined functions are a LINE FORM handled before this
   grammar: DEF name(param [= default], ...) { body } -- the body is
   newline/-separated commands using the parameters as variables;
   each body line must itself satisfy this grammar. Invocation uses
   the ordinary call syntax name(arg, ...); trailing parameters
   take their defaults. *)
init   := IDENT "=" expr ;
path   := IDENT { "." IDENT } ;
                                (* objN.field[.x|y|z|w],
                                system.field,
                                contactK.field,
                                name.field for
                                AS-registered names *)

expr   := term { ("+" | "-") term } ;
term   := unary { ("*" | "/" ) unary } ;
unary  := "-" unary | atom ;
atom   := NUMBER | STRING | "[" expr { "," expr } "]" | "(" expr ")"
      | IDENT "(" [ expr { "," expr } ] ")" (* builtin or user
                                             function call *)
      | path
      | IDENT ;
                                (* parameter / LET var *)

```

Operator precedence is the usual one: `*` and `/` bind tighter than `+` and `-`; unary minus binds tightest; parentheses override everything.

One subtlety worth learning early: GET takes *only a path*. To do arithmetic with a field, use a *bare expression*:

```

get obj0.position.x - 5      # ERROR -- GET wants just a path
obj0.position.x - 5         # OK -- bare expression

```

A second subtlety — SCENE arguments are *terms*, not full expressions. The scene commands take several numbers in a row with no commas between them. If those numbers were parsed as full expressions, `scene rotate 15 -5` would be read as *one* argument $15 - 5 = 10$ (whitespace never matters to the lexer) and the command would then be missing its second argument. So scene arguments stop one level lower in the grammar, at **term**: `-5` is negative five, `2*2` and `1/3` still work, but a sum or difference must be parenthesized:

```

scene rotate 15 -5           # yaw +15 deg, pitch -5 deg (two arguments)
scene zoom (1 + 0.5)         # a sum needs parentheses
scene translate 2*2 0 1/2    # * and / are fine unparenthesized

```

A third subtlety — bare identifiers compile and resolve at execution. An IDENT alone is an ordinary atom: the constants `pi` and `tau`, a function parameter, or a LET variable (§5.9). The name is looked up when the line *runs*, not when it parses, so a function body may use parameters

freely; a name that is bound to nothing fails at execution with an error that lists the three ways to bind one (LET, a parameter, or a registered object's `name.field` path). A call `IDENT(...)` is a builtin (`dot`, `cross`, `norm`, `normalize`, `sqrt`, `abs`, `sin`, `cos`, `exp`, `log`) when the name matches one, and a user-defined function otherwise.

4 The type system (what values exist)

Every expression evaluates to one of these *values*:

type	written as	notes
number	2, -0.5, 1e-3	64-bit float
vec3	[1, 2, 3]	exactly 3 numeric entries
quaternion	[w, x, y, z]	exactly 4 numeric entries, w first ; assigned to orientation it is renormalized to unit length
mat3	[[a,b,c],[d,e,f],[g,h,i]]	3 rows of 3 numbers
string	"ball"	a double-quoted literal (§2.1) — names objects (NEW ... AS) and feeds user functions; also produced as results like <code>obj0</code> or run summaries

Bracket literals are *shape-directed*: 3 numbers make a `vec3`, 4 numbers make a quaternion, 3 `vec3`s make a `mat3`.

Arithmetic is *type-checked*:

operation	allowed
+, -	number±number, vec3±vec3, quat±quat
*	number·number, number·vec3, number·quat, number·mat3, mat3·vec3, mat3·mat3, quat·quat (Hamilton product)
/	number/number, vec3/number
vec3*vec3	forbidden — the error tells you to use <code>dot()</code> or <code>cross()</code>

Builtins: `dot(a,b)`, `cross(a,b)`, `norm(v)` (length; also quaternions and numbers), `normalize(v)`, and scalar `sqrt` `abs` `sin` `cos` `exp` `log`. Constants: `pi`, `tau` ($= 2\pi$).

4.1 Special functions

The same call syntax reaches the `special_functions` library. Nothing about the grammar changes to accommodate them — the production `IDENT "(" [expr {" ," expr}] ")"` already admits any builtin — so adding a function is a *registration*, not a grammar change.

Orders must be whole numbers. Where an argument is an integer order (the n , ℓ , m below), a fractional value is an error rather than being quietly truncated:

```
In[1]:= hermite_h(2.5, 1)
Err[1]: hermite_h(): argument 1 must be a whole number (an integer order), got 2.5
```

That refusal is deliberate. Truncating to `hermite_h(2, 1)` would return a confident, wrong number, and you would have no way to notice.

family	functions
spherical Bessel	sph_j(n,x), sph_y(n,x), sph_j_prime(n,x), sph_y_prime(n,x)
Legendre	legendre_p(n,x), legendre_p_prime(n,x), assoc_legendre_p(l,m,x), norm_assoc_legendre_p(l,m,x)
spherical harmonics	sph_harm(l,m,theta,phi) → [re, im], sph_harm_real(l,m,theta,phi)
orthogonal polynomials	hermite_h(n,x), hermite_he(n,x), laguerre_l(n,x), laguerre_l_assoc(n,alpha,x), chebyshev_t(n,x), chebyshev_u(n,x), gegenbauer_c(n,alpha,x), jacobi_p(n,alpha,beta,x)
cylindrical Bessel	bessel_j(n,x), bessel_j_array(n_max,x) → list
quadrature	gauss_legendre(n) → [nodes, weights]
eigenproblems	eigenvalues(matrix) → list, jacobi_eigen(matrix) → [values, vectors]
angular momentum	wigner_3j(j1,j2,j3,m1,m2,m3), wigner_6j(j1,j2,j3,j4,j5,j6), clebsch_gordan(j1,m1,j2,m2,j3,m3)
linear algebra	solve_tridiag(sub, diag, sup, rhs) → list, solve_tridiag_c(...) → list, solve_cyclic_tridiag_c(sub, diag, sup, rhs, bl, tr) → list
utility	rel_err(a, b)

Angular momenta may be half-integers, so `wigner_3j`, `wigner_6j` and `clebsch_gordan` take plain numbers rather than demanding whole ones — `clebsch_gordan(0.5, 0.5, 0.5, -0.5, 1, 0)` is exactly what you want to write for two spin- $\frac{1}{2}$ particles. A coupling that violates a selection rule returns **0**, which is the mathematically correct answer, not an error; a value that is not an angular momentum at all ($j = 0.3$, or a negative j) *is* an error.

A wrinkle worth knowing about lists. The bracket literal is overloaded: `[a,b,c]` is a *vector*, `[a,b,c,d]` is a *quaternion*, and any other length is a list. That is convenient for physics and would be a nuisance here, so all three shapes are accepted anywhere a numeric list is expected — a 4×4 matrix whose rows are 4-element brackets works exactly as you would expect, and `norm_assoc_legendre_p` never lectures you about quaternions. A 3×3 matrix value is likewise accepted directly wherever a matrix argument is wanted.

4.2 Complex numbers

A number with an `i` suffix is imaginary, so `2 + 3i` needs no complex literal syntax of its own — it is ordinary addition of a real and an imaginary:

```
In[1]:= 3i
Out[1]= 0 + 3i
In[2]:= 2 + 3i
Out[2]= 2 + 3i
In[3]:= (2 + 3i) * (2 - 3i)
Out[3]= 13 + 0i
In[4]:= 1 / (0 + 1i)
```

```
Out[4]= 0 - 1i
```

`+`, `-`, `*` and `/` all accept complex operands, and reals promote automatically. Note `In[3]`: the result stays typed as complex and displays `13 + 0i` rather than collapsing to `13`. That is deliberate — the type you get out should not depend on whether a particular cancellation happened to be exact.

The suffix only binds when the `i` is not followed by another identifier character, so `2intercept` still lexes exactly as it did before complex numbers existed.

This is what makes the complex Crank–Nicolson solvers reachable. `solve_tridiag_c` and `solve_cyclic_tridiag` take the same arguments as the real solver and accept a mix — a real band with a complex diagonal and a real right-hand side is fine, since reals promote:

```
In[5]:= solve_tridiag_c([0, 1, 1], [1i, 1i, 1i], [1, 1, 0], [1, 0, 0])
Out[5]= [0 - 0.6666666666666666i, 0.33333333333333337 + 0i, 0 + 0.3333333333333333i]
```

Over the machine bridge (JupyterLab), a complex value is reported as the two-element array `[re, im]`, matching how `sph_harm` reports its value — JSON has no complex type.

5 Command semantics

5.1 NEW — create an object

```
NEW POINT      { field = expr, ... }
NEW SPHERE     { field = expr, ... }
NEW CUBOID     { field = expr, ... }      (alias: NEW CUBE)
NEW TORUS      { field = expr, ... }
NEW DISK       { field = expr, ... }      (alias: NEW DISC)
NEW CYLINDER   { field = expr, ... }
NEW DUMBBELL   { field = expr, ... }      (alias: NEW DUMBBELL)

NEW <shape> AS <name> { field = expr, ... }  (register a user name)
```

Creates one *physical object* and prints its handle (`obj0`, `obj1`, ..., numbered by position). Defaults: mass 1, at the origin, at rest, no charge; sphere radius 1; cuboid half-extents `[1,1,1]`; torus `ring_radius` 1, `tube_radius` 0.25; disk radius 1; cylinder radius 0.5, `half_height` 1; dumbbell `m1` = 1, `m2` = 1, `m_rod` = 0.5, `r1` = 0.25, `r2` = 0.25, `rod_radius` = 0.1, `length` = 1 (total mass 2.5).

Initializer fields: `mass`, `charge`, `position`, `velocity`, `momentum`, `orientation`, `angular_velocity`, `angular_momentum`, `radius` (spheres, disks, cylinders), `half_extents` (cuboids), `ring_radius`, `tube_radius` (tori — or the `inner_radius` + `outer_radius` pair, see below), `height`, `half_height` (cylinders; `HEIGHT` is the full height = `2·half_height`), `m1`, `m2`, `m_rod`, `r1`, `r2`, `rod_radius`, `length` (dumbbells, see below), `inertia_tensor`, `inverse_inertia_tensor`, `magnetic_moment_tensor`, `force`, `torque`, `id`, `inverse_mass`.

A torus can be sized two ways: directly (`ring_radius` c = the radius of the circle traced by the tube’s center, `tube_radius` a = the tube’s own radius) or by the **inner_radius** + **outer_radius** pair (the hole’s radius and the outermost radius: $\text{inner} = c - a$, $\text{outer} = c + a$). Inside a `NEW` initializer list the torus geometry is **deferred**: the four radius fields are collected and resolved *once*, at the

end of the list — `ring_radius/tube_radius` are applied first, then `inner_radius/outer_radius` override the derived pair — and the result is validated once, against the *final* values (a bad pair fails with `torus needs 0 <= inner < outer (got inner = 2, outer = 1)`). Giving both members of the pair is therefore **genuinely order-independent**: `{ inner_radius = 1, outer_radius = 2 }` yields ring 1.5, tube 0.5 whichever comes first, and a pair that shrinks the default torus — `{ outer_radius = 0.5, inner_radius = 0.2 }` — works in either order too (checking each write against the half-updated default would have refused one of the two orders). `inner_radius = 0` is legal — that is the **horn torus**, whose tube touches the axis — on **NEW** and on **SET** alike.

The **dumbbell** is ONE rigid body: two solid spheres joined by a solid rod. Its seven part fields are `m1`, `m2`, `m_rod` (the two sphere masses and the rod mass), `r1`, `r2` (the sphere radii), `rod_radius` (alias `rod_r`) and `length` (alias `len` — the distance between the sphere centers). Like the torus radius pair, the part fields are **deferred**: they are collected during the initializer list and resolved *once* at its end, so they are order-independent and validated once against the final values. The constructor places the local origin at the composite **center of mass** — the sphere centers sit at $z_1 = -(m_2 + m_{\text{rod}}/2) L/M$ and $z_2 = +(m_1 + m_{\text{rod}}/2) L/M$ on the body z -axis, which makes $m_1 z_1 + m_2 z_2 + m_{\text{rod}}(z_1 + z_2)/2 = 0$ an identity — so **position** is the COM, exactly as for every other shape. The inertia tensor is the exact composite (solid-sphere $\frac{2}{5}mr^2$ terms with parallel-axis shifts, plus the rod), and the surface is the exact union of the three parts. It is the simulator's first non-centrally-symmetric shape; collisions against every other shape decompose exactly over its parts (see `collision_detection.pdf`). The part fields remain readable *and writable* after creation (§5.2).

AS registers a user name for the object. The name is a string literal (`NEW SPHERE AS "ball"`) or a bare identifier (`NEW SPHERE AS ball`); a bare identifier is first resolved against a function parameter or LET variable holding a string — so a function can name the object it creates from its argument (§5.9, Example 14) — and otherwise the identifier's own spelling is the name. The reply shows both handles: `obj0 as ball`. Named paths then work everywhere a positional path does — `ball.mass`, `dumbell0.position.x`, in **SET/GET** and in bare expressions. Names are case-insensitive identifiers; `system/sys` and the positional spellings are refused (`'system'` is reserved, `'obj7'` is reserved for positional paths), and so are duplicates (the name `'d'` already refers to `obj0` -- DEL it or pick another name). The registry follows every renumbering: DEL removes the deleted object's name and shifts the names of higher-numbered objects down with their objects, and the **BOX** commands do the same when walls come and go.

Four guarantees make initializers forgiving:

1. **Order does not matter for velocities.** `velocity` and `angular_velocity` are applied *after* mass and inertia are final, so `{ velocity = [1,0,0], mass = 2 }` still yields momentum `[2,0,0]`.
2. **Inertia is computed for you.** For every extended shape the inertia tensor is recomputed from the final mass and shape (solid sphere $\frac{2}{5}mr^2$; cuboid $\frac{m}{3}(h_y^2 + h_z^2)$ etc.; torus $I_z = m(c^2 + \frac{3}{4}a^2)$, $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$; disk $I_z = \frac{1}{2}ma^2$, $I_{xy} = \frac{1}{4}ma^2$; cylinder $I_z = \frac{1}{2}mr^2$, $I_{xy} = m(3r^2 + 4h^2)/12$ with h the half-height) — *unless* you supplied `inertia_tensor` yourself, in which case yours is kept.
3. **Coupled fields stay consistent** (see **SET** below).
4. **NEW is transactional.** If any initializer — or the final geometry validation — fails, the half-built object is removed before the error is reported: a failing **NEW** never leaves a ghost

behind, and `system.count` and the `objN` numbering are exactly what they were before the command.

5.2 SET / GET — write and read any field

A *path* is `objN.field`, `system.field`, `contactK.field` (§5.7) or `name.field` for a name registered with `NEW ... AS` (§5.1), optionally followed by a component `.x .y .z` (vectors) or `.w .x .y .z` (quaternions). Component writes are safe read-modify-write operations through the full field's get/set pair. Every object also has six **component shorthands**: `.x .y .z` read and write the position components and `.vx .vy .vz` the velocity components — `ball.x` is `ball.position.x`, `SET ball.vx = 2` is a velocity write.

Object fields (R = readable, W = writable):

field	type	R/W	meaning
<code>id</code>	number	RW	user label (no effect on physics)
<code>mass</code>	number	RW	writing also updates <code>inverse_mass</code> ($m \leq 0 \Rightarrow 1/m := 0$)
<code>inverse_mass</code>	number	RW	writing back-computes <code>mass</code> ; 0 makes the body static
<code>charge</code>	number	RW	Coulombs
<code>position, pos</code>	vec3	RW	world position
<code>velocity, vel</code>	vec3	RW	<i>derived</i> : reads p/m , writes $p := mv$
<code>momentum</code>	vec3	RW	canonical stored linear state
<code>orientation</code>	quat	RW	renormalized on write
<code>angular_velocity</code>	vec3	RW	<i>derived</i> : $\omega = R I^{-1} R^T L$; writes $L := R I R^T \omega$
<code>angular_momentum</code>	vec3	RW	canonical stored angular state (spin)
<code>inertia_tensor</code>	mat3	RW	body frame; write updates the inverse (singular $\rightarrow$ zero inverse = cannot rotate)
<code>inverse_inertia_tensor</code>	mat3	RW	write back-computes the tensor
<code>magnetic_moment_tensor</code>	mat3	RW	torque $\tau = (R M R^T) B$
<code>radius</code>	number	RW	spheres, disks, cylinders; write re-computes inertia and keeps the shape family (a disk stays a disk, a cylinder keeps its height); a torus is refused — <code>obj6</code> is a torus - <code>set ring_radius/tube_radius</code> or <code>inner_radius/outer_radius</code> instead of <code>radius</code> — rather than silently becoming a sphere
<code>half_extents</code>	vec3	RW	cuboids only; write recomputes inertia
<code>ring_radius, tube_radius</code>	number	RW	tori only; write recomputes inertia

<code>inner_radius, outer_radius</code>	number	RW	tori only, derived ($\text{inner} = c - a$, $\text{outer} = c + a$); writing one holds the other fixed; <code>inner_radius</code> accepts ≥ 0 (0 = the horn torus; the other radii must be > 0)
<code>height, half_height</code>	number	RW	cylinders only; <code>height</code> is the full height ($= 2 \cdot \text{half_height}$); write recomputes inertia
<code>m1, m2, m_rod</code>	number	RW	dumbbells only: the two sphere masses and the rod mass; writing rebuilds the body — total mass, the COM offsets and the inertia tensor all follow (position is untouched; the stored momenta are preserved — momentum-canonical, like every mass write — so the velocities rescale with the new mass and inertia)
<code>r1, r2, rod_radius</code>	number	RW	dumbbells only: the sphere radii and the rod radius (alias <code>rod_r</code>); same rebuild on write
<code>length</code>	number	RW	dumbbells only: distance between the sphere centers (alias <code>len</code>); same rebuild on write
<code>x, y, z</code>	number	RW	every object: shorthands for <code>position.x/.y/.z</code>
<code>vx, vy, vz</code>	number	RW	every object: shorthands for <code>velocity.x/.y/.z</code>
<code>boundary, shape</code>	string	R	description of the shape
<code>kinetic_energy, energy</code>	number	R	$\frac{1}{2}m v ^2 + \frac{1}{2}\omega \cdot L$
<code>force</code>	vec3	RW	constant external force during STEP/RUN
<code>torque</code>	vec3	RW	constant external torque
<code>restitution</code>	number	RW	collision bounciness $e \in [0, 1]$ (default 1 = elastic; a pair uses $\min(e_i, e_j)$)

System fields:

field	type	R/W	meaning
<code>g_constant, g</code>	number	RW	Newton's G (default 1)
<code>softening</code>	number	RW	Plummer length ε (default 10^{-6}); forces use $(r^2 + \varepsilon^2)^{3/2}$
<code>uniform_gravity, gravity</code>	vec3	RW	constant acceleration field
<code>e_field</code>	vec3	RW	uniform E ; force qE
<code>b_field</code>	vec3	RW	uniform B ; force $qv \times B$, torque $(RMR^T)B$
<code>rtol, atol</code>	number	RW	CVODE tolerances (10^{-10} , 10^{-12})
<code>time, t</code>	number	RW	current simulation time
<code>method</code>	string	R	current integrator
<code>count, n</code>	number	R	number of objects
<code>collide</code>	number	R	1 when collision detection is on (switch with COLLIDE ON/OFF)
<code>contacts</code>	number	R	contacts recorded by the last STEP/RUN
<code>collisions</code>	number	R	running total of resolved impulses
<code>restitution_threshold</code>	number	RW	approach speeds below this bounce with $e = 0$ (default 10^{-3})
<code>contact_slop</code>	number	RW	tolerated overlap before positional projection (default 10^{-9})
<code>box</code>	number	R	inner side length of the rigid bounding box (§5.8); 0 = none. Create/remove with BOX

5.3 STEP and RUN — integrate (always via SUNDIALS)

STEP `dt` advances time by `dt`. RUN `t STEPS n` advances by `t`, stopping at `n` evenly spaced output points (default 10). Both accept full expressions (`run 2 * pi steps 8` is legal). The reply summarizes the run; `|dE/E|` is the relative change in total energy — your built-in sanity check. **All integration is performed by the pure-Rust SUNDIALS solvers**; there is no hand-rolled stepper.

5.4 METHOD — choose the integrator

METHOD ADAMS	(default; CVODE Adams-Moulton, adaptive)
METHOD BDF	(CVODE BDF -- stiff problems, fast gyration)
METHOD SPRK <table> [dt]	(ARKODE symplectic, fixed step; default dt 0.01)

Table names may be abbreviated: `leapfrog_2_2` becomes `ARKODE_SPRK_LEAPFROG_2_2`. Useful tables: `EULER_1_1`, `LEAPFROG_2_2` ($\equiv$ classic velocity Verlet), `MCLACHLAN_2_2/3_3/4_4/5_5_6`, `RUTH_3_3`, `YOSHIDA_6_8`.

SPRK requires a *separable* system: point-like translation only. If anything velocity-dependent or rotational is active (a `b_field`, a magnetic tensor, an external torque, a spinning rigid body), RUN refuses with an error *naming the offending feature*.

5.5 Observables, bookkeeping, help

command	prints
ENERGY	kinetic + softened pairwise potential + uniform-field potentials
COM	system center of mass
MOMENTUM	total linear momentum
ANGMOM	total angular momentum about the origin (orbital + spin)
LAPLACE n	Laplace–Runge–Lenz vector of object n about the center of mass, $k = G M_{\text{total}}$
LIST	one line per object
DEL n	removes object n (later objects renumber!)
RESET	wipes everything (not to be confused with SCENE RESET, which re-initializes only the scene window’s playback copy — see <code>scene_info.pdf</code>)
HELP	the quick-reference card

5.6 SCENE — the graphical scene window

SCENE CREATE starts a tiny web server *inside* posim (pure Rust standard library — no external dependencies) and opens a page in your web browser. That page **is** the scene window: it draws every simulator entity on a 3-D canvas, has a **toolbar** (Start, Pause, Stop, Reverse, a permanent Reset button, single-step, a dt box, zoom, view reset, grid/trails/labels toggles, help) and a **status bar** (connection light, playback mode, simulation time, dt, total energy, body count, hidden count, history depth, camera readout, frames per second). Inside the window:

gesture	effect
← → (arrow keys)	translate the view left / right
↑ ↓	translate the view up / down
left-click + drag	rotate (orbit) around the scene
mouse wheel	zoom in / out
+ / -	zoom in / out from the keyboard
shift-drag or right-drag	translate (pan) with the mouse
Space	start / pause playback
R, G, T, L, H	reset view, toggle grid / trails / labels, help

The same controls are scriptable from the notebook:

command	effect
SCENE CREATE [port]	open the window (all entities shown). Default port: one chosen by the OS; give a number (0–65535) to pin it. Prints the URL. A second CREATE is harmless — it just reminds you of the URL.
SCENE CLOSE (= DESTROY)	shut the server down; every window disconnects
SCENE TRANSLATE dx dy [dz]	move the camera’s look-at point by (dx, dy, dz) world units (dz defaults to 0)
SCENE ROTATE dyaw dpitch	orbit the camera: yaw (azimuth) and pitch (elevation) in degrees ; pitch is clamped to $\pm 89^\circ$
SCENE ZOOM IN / OUT / f	zoom by 1.25 $\times$, by 1/1.25, or by any factor $f > 0$ ($f > 1$ zooms in)

command	effect
SCENE HIDE [n ALL]	hide object n, or everything (bare HIDE = HIDE ALL)
SCENE SHOW [n ALL]	undo HIDE
SCENE REFRESH	copy the notebook's current system into the window (see below) and clear playback history
SCENE REDRAW	re-send the complete scene description to every window (forces a full redraw)
SCENE START	begin time-stepped evolution, forward
SCENE PAUSE	freeze; START resumes, history is kept
SCENE STOP	halt and clear the recorded history
SCENE REVERSE	play backward in time through the recorded history
SCENE RESET	re-initialize the playback: every mutable value and the time return to their initial values — the state last synced at CREATE/REFRESH, restored bit-identically — history and the step counter clear, and the mode returns to <i>stopped</i> ; START then re-runs the simulation from the beginning. The window's permanent toolbar Reset button does exactly the same (both call one primitive)
SCENE SET_TIME_STEP dt	set the playback time step (must be positive and finite)
SCENE STATUS	a four-line report: URL + connected windows; mode/t/dt/steps/history; entities + hidden list; camera print (and clear) the asynchronous messages the window has sent: errors, connect/disconnect notices, toolbar actions, data requests
SCENE EVENTS	

The playback copy. The window animates its *own synchronized copy* of your system, evolved by a background thread — your notebook state does **not** move while the animation runs, so `GET obj0.position` still answers instantly and exactly. The copy is taken at **CREATE** and again at every **REFRESH**. All forward stepping goes through the same SUNDIALS integrators as **STEP/RUN** — there is no separate physics engine in the window.

Playback is a four-state machine (*stopped* $\rightarrow$ *running* $\rightleftharpoons$ *paused*, plus *reversing*): **STOP** clears history and a later **START** begins fresh; **PAUSE** keeps history so both **START** (forward) and **REVERSE** (backward) can continue from the freeze point. **RESET** is stronger than **STOP**: besides clearing history it puts every value back to its initial state, so the next **START** replays the simulation from the very beginning rather than continuing from wherever playback halted. The reply confirms it:

```
In[6]:= scene reset
Out[6]= scene playback reset to its initial state (t = 0, 1 entity); Start runs the
simulation again from the beginning
```

How REVERSE works. While running forward, the playback thread records a snapshot of the whole system before every step (a ring buffer, at most 20 000 frames). **REVERSE** replays those snapshots newest-first, which is an *exact* rewind — bit-for-bit the states you already visited. When the buffer runs out it pauses and sends an event (**reverse: reached the beginning of recorded history -- paused**). Reversing with no history at all is refused with an error.

Asynchronous events. The window can talk back at any time — it reports JavaScript errors,

connections, disconnections, and every toolbar action. These messages queue up inside `posim`; `SCENE EVENTS` drains the queue (up to 1000 messages are kept). In JupyterLab they also appear *by themselves*, without you asking — see §7.

5.7 COLLIDE and CONTACTS — rigid-body collisions

COLLIDE	(report: on/off, collidable pairs, impulses so far)
COLLIDE ON	(default - collisions are detected in EVERY scene)
COLLIDE OFF	(pure point-mass/gravity studies)
CONTACTS	(list every contact of the last STEP/RUN)

When collisions are on (the default) and the system contains at least one collidable pair (spheres, cuboids, or a point against either — two points cannot collide), `STEP` and `RUN` detect impacts **during** the time step by SUNDIALS *event rootfinding*: the integrator itself lands on the instant where a pair’s signed separation crosses zero, interpolated to solver precision. At that instant an impulse acts along the **contact normal** — the line of the action–reaction force pair — and integration continues. Nothing tunnels, and the reply reports what happened.

Every contact of the last `STEP`/`RUN` is recorded and exposed to the rest of the simulation through read-only `contactK` paths:

field	type	meaning
<code>contactK.i</code> , <code>contactK.j</code>	number	the colliding pair (indices, $i < j$)
<code>contactK.t</code>	number	the event time (the root the solver landed on)
<code>contactK.point</code>	vec3	world-space contact point
<code>contactK.normal</code>	vec3	unit normal, from <code>obji</code> toward <code>objj</code> — the action–reaction line; <code>objj</code> receives $+J\hat{n}$, <code>obji</code> receives $-J\hat{n}$
<code>contactK.depth</code>	number	penetration depth at the event (≈ 0)
<code>contactK.rel_vel_n</code>	number	approach speed along the normal (negative = approaching)
<code>contactK.impulse</code>	number	scalar impulse magnitude J

Response physics (full derivation in `collision_detection.pdf`): per-object **restitution** $e \in [0, 1]$ (1 = elastic default, 0 = perfectly plastic), pair-combined as $\min(e_i, e_j)$; impulse $J = -(1 + e)(\mathbf{v}_{\text{rel}} \cdot \hat{n}) / (\hat{n}^\top K \hat{n})$ with the effective-mass matrix K including the angular terms, applied through the canonical momentum/angular-momentum setters; static bodies (`inverse_mass` = 0) are immovable walls; approach speeds below `system.restitution_threshold` settle without bounce; leftover overlap beyond `system.contact_slop` is projected out. `SCENE` windows draw each contact normal as an arrow at the contact point (**Contacts** toolbar button or the `C` key).

5.8 BOX — the rigid bounding box

BOX <size>	(create: six static walls enclosing an inner size x size x size cube centered on the origin)
BOX OFF	(remove the box - its six walls are deleted)
BOX	(bare: report status)

`BOX <size>` builds a closed rigid room out of **six ordinary cuboid objects** — wall slabs behind the planes $x = \pm \text{size}/2$, $y = \pm \text{size}/2$, $z = \pm \text{size}/2$ (half-thickness `size/4`, centers at $\pm 3 \cdot \text{size}/4$, cross-sections wide enough to cover the corners) — and prints their handles:

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
        obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
        the inside faces
```

Infinite mass is exact, not approximate. The equations of motion never use the mass itself — only the *inverse* mass: velocity is $v = pm^{-1}$, and the collision impulse divides by $\hat{n}^\top K \hat{n} = m_i^{-1} + m_j^{-1} + (\text{angular terms})$ (§5.7). Each wall is created with `inverse_mass = 0` and a zero inverse inertia tensor, so it contributes exactly 0 to every impulse denominator and receives no state writes: bodies bounce off the inside faces elastically while the walls stay **bit-identically at rest**. One measurable consequence: system **momentum is not conserved** inside a box — the infinitely massive walls absorb it without moving (Example 13). LIST tags every wall [`wall: static, inverse_mass=0`] (and shows `mass=0` — the canonical stored quantity for a static body is the inverse).

Bookkeeping:

- `GET system.box` reads the inner side length; 0 means no box. The path is read-only — the box is created and removed by the command.
- A second `BOX <size>` **replaces** the existing box: the old walls are removed first and the reply notes (replacing the previous box).
- The walls are ordinary objects with ordinary indices: `DEL` on a wall deletes it like any other object **and dissolves the box** — `system.box` drops to 0 (five walls no longer enclose anything) — but the surviving slabs **stay tracked**: LIST keeps their [`wall: static, inverse_mass=0`] tag, and bare `BOX` reports `box: dissolved` (a wall was deleted; 5 tracked slab(s) remain — `BOX <size>` replaces them, `BOX OFF` removes them). The next `BOX <size>` removes the survivors *before* building the new box (the reply notes (replacing the previous box)) and `BOX OFF` simply deletes them — either way no orphan slab leaks. Wall indices are tracked through every `DEL` renumbering, so deleting a non-wall object never confuses the box.
- `BOX OFF` replies `box removed` (6 wall(s) deleted, indices renumbered) — or `box: none` if there was nothing to remove; bare `BOX` reports the size and wall handles, or `box: none` (`BOX <size>` creates one).
- A `SCENE` window draws the box as a **dashed interior wireframe**; the wall slabs themselves are not drawn as bodies. Creating a box while a window is open appends a reminder to the reply: (`scene window open: SCENE REFRESH` shows the box). `RESET` re-syncs an open window to the now-empty system **and clears its box wireframe and wall flags** — no stale overlay survives the wipe.

5.9 User definitions: DEF, LET, FUNCS, SHOW

<code>DEF name(param [= default], ...) { body }</code>	(define a function)
<code>name(arg, ...)</code>	(call it)
<code>LET name = <expr></code>	(session variable)
<code>FUNCS</code>	(list signatures; alias: <code>FUNCTIONS</code>)
<code>SHOW name</code>	(print a function's source)

DEF is a line form, not a grammar production. A line that starts with DEF is recognized *before* the ordinary grammar (§3): the notebook captures everything up to the closing } — interactively it keeps prompting ...:= for continuation lines until the brace closes; scripts and JupyterLab cells just keep reading — and installs the function. The **body** is a sequence of ordinary commands, separated by newlines or ;, in which the parameters act as variables. **Every body line is syntax-checked at definition time** (a typo fails the DEF immediately, naming the body line), and **defaults are ordinary expressions evaluated once, at definition** (LET variables are visible in them).

Calling uses the ordinary call syntax `name(arg, ...)` — the same atom as `sqrt(2)`; a name that is not a builtin is looked up among your functions. Missing trailing arguments take their defaults (an argument with no default must be supplied). Each call pushes a **call frame** binding the parameters (depth cap: 32 — the language has no conditionals, so recursion could never terminate anyway) and runs the body lines through the same compile-and-execute pipeline as typed input. The call **returns the last body line's value** — if that line is a SET, the call prints no Out[n], exactly like SET itself. A failing body line aborts the call and the error names the function *and* the line; lines that already ran keep their effects (each command's own guarantees still hold per line — e.g. a failing NEW inside a body is still transactional), so a partly-run call leaves no half-built object, only completed commands.

Editing is SHOW + re-DEF. SHOW `name` prints the definition verbatim, exactly as you typed it; redefining the same name replaces the old version and the reply appends (**redefined**). FUNCS lists every signature with its defaults.

LET name = expr binds a session variable (reply: `name set`). It is visible in bare expressions, in function bodies, in DEF defaults — and, holding a string, it can *name* things: NEW ... AS and named paths resolve a bare identifier through the parameter/LET bindings first (§5.1). Note GET takes only a *path* (§3), so a variable is read back as a bare expression: `g0`, not `GET g0`.

A complete session (genuine output):

```
In[1]:= let g0 = 9.81
Out[1]= g0 set
In[2]:= def drop(name, m = 1, h = 10) {
  new sphere as name { mass = m, position = [0, h, 0] }
  set system.gravity = [0, -g0, 0]
}
Out[2]= function drop(3 parameter(s)) defined -- 2 body line(s)
In[3]:= drop("ball")
In[4]:= get ball.position.y
Out[4]= 10
In[5]:= drop("pebble", 0.1, 2)
In[6]:= get pebble.mass
Out[6]= 0.1
In[7]:= funcs
Out[7]= drop(name, m = 1, h = 10) -- 2 body line(s); SHOW drop prints it
In[8]:= show drop
Out[8]= def drop(name, m = 1, h = 10) {
  new sphere as name { mass = m, position = [0, h, 0] }
  set system.gravity = [0, -g0, 0]
}
In[9]:= def drop(name, m = 1, h = 100) { new sphere as name { mass = m, position = [0,
```

```

    h, 0] }; set system.gravity = [0, -g0, 0] }
Out[9]= function drop(3 parameter(s)) defined -- 2 body line(s) (redefined)
In[10]:= drop("skydiver")
In[11]:= skydiver.y
Out[11]= 100
In[12]:= speed
Err[12]: unknown name 'speed' (define it with LET, pass it as a function parameter, or
        use 'speed.field' for a registered object)

```

What to notice. The function's **name** parameter holds a string, and **new sphere** as **name** inside the body names each created object from it — **ball**, **pebble**, **skydiver** — so one definition mints a family of named objects. **drop("ball")** used both defaults; cell 9 is the edit loop (copy cell 8's **SHOW** output, change **h**, re-DEF — note **;** separating body lines works as well as newlines); calls 3, 5 and 10 print no **Out** because the last body line is a **SET**. Cell 12 is the bare-identifier error: execution-time resolution means the message can list every way to bind a name.

Definition-time and call-time errors are specific: a reserved or builtin name is refused (**DEF: 'norm' is a builtin function and cannot be redefined**), nested **DEF** is refused, an empty body is refused, a missing non-default argument fails as **name(): missing argument 'p' (it has no default)**, too many arguments fail naming the signature, and the depth cap fails as **function call depth limit (32) exceeded**.

6 The notebook (cells, editing, magics)

6.1 Cells

Start **posim** with no arguments. You get numbered prompts, exactly like Jupyter or Mathematica:

```

In[1]:= new sphere { mass = 2, radius = 0.5 }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[0.2, 0, 0], [0, 0.2, 0], [0, 0, 0.2]]

```

Pressing **Enter** executes the cell — in a plain terminal, Enter *is* shift-enter. Commands with no interesting result (like **SET**) print no **Out[n]**. Errors print **Err[n]:** and the session continues.

6.2 State is cumulative

Like Jupyter, the notebook has one live simulator state that every cell mutates in order. Re-running an old **NEW** cell creates a *new* object; it does not replace the old one. To rebuild cleanly, **%reset** and replay.

6.3 Magics

magic	effect
%history	lists every cell with input and output; failed cells marked !
%edit n <text>	replaces cell <i>n</i> 's input and executes it as the next cell
%rerun n	executes cell <i>n</i> 's input again as a new cell
%save <file>	writes all successful non-magic inputs to a replayable script
%load <file>	replays a script file cell by cell
%reset	clears the simulator (history kept)
%quit, %exit	leave

A plain terminal has no cursor-addressable cells (posim uses only the Rust standard library — no raw terminal mode), so backward/forward movement is by these magics. **For true click-and-edit cells with shift-enter, use the JupyterLab front end (§7).**

6.4 Scripts

`posim -script file` executes a file of command lines (blank lines and `#` comments skipped), echoing `In[n]/Out[n]`, and exits nonzero if any cell failed — good for CI.

`posim -notebook file` executes the same file format but then **stays in the interactive notebook** instead of exiting: the loaded cells keep their `In[n]` numbers, the next command continues the numbering, and a scene window the file opened (`SCENE CREATE`) stays alive — the launcher for the *dynamic notebooks* in `dynamic_notebooks/`, files that build a simulation and open the GUI ready for Start. A failing cell aborts the load with a nonzero exit instead of entering the notebook.

7 JupyterLab in one paragraph

The `jupyter/` directory ships a small *wrapper kernel*: JupyterLab starts a Python shim, the shim starts `posim -machine` (a JSON request/reply protocol over stdin/stdout), and each notebook cell you shift-enter is forwarded line-by-line as `{"op": "exec", "code": ...}`. Setup and tests: `jupyter/README.md`. Everything in §§2–5 applies verbatim inside JupyterLab cells; multi-line cells run one line at a time, stopping at the first error.

8 The stack machine (what runs your line)

The parser emits *postfix instructions*; e.g. `set obj0.mass = 2 + 3 * 4` compiles to

```
Push 2
Push 3
Push 4
Mul          <- pops 3,4  pushes 12
Add          <- pops 2,12 pushes 14
Store obj0.mass <- pops 14, calls set_mass(14)
```

The machine's whole memory is a stack of typed values. Instructions: `Push`, `Load`, `Store`, `LoadIdent` (a bare identifier — parameter or LET variable, resolved at execution, §5.9), `StoreGlobal` (LET), `Add` `Sub` `Mul` `Div` `Neg`, `PackList` (literals), `Call` (builtin or user function — a user call runs

each body line through this same compile-and-execute pipeline inside a call frame), `NewObject/InitField/FinishM` (which also registers the AS name), `Delete`, `ListObjects`, `ListFns/ShowFn` (`FUNCS/SHOW`), `Step`, `Run`, `SetMethod`, `Energy`, `CenterOfMass`, `TotalMomentum`, `TotalAngularMomentum`, `Laplace`, `Reset`, `Help`. `Load/Store` go *only* through the `physical_object` get/set API — the machine cannot corrupt simulator state, and every coupled invariant (`mass↔inverse`, `inertia↔inverse`, unit quaternions) is enforced on every write. When the program ends, the top of the stack becomes `Out[n]`.

9 Fifteen worked examples

All transcripts are genuine program output.

9.1 Example 1 — an eccentric Kepler orbit and the Runge–Lenz compass

The Laplace–Runge–Lenz vector **A** points along a Kepler orbit’s major axis and is conserved *only* for a perfect $1/r^2$ force — the most delicate conservation test available. We build a “sun” so heavy the reduced problem is exact and integrate two orbits with a 4th-order *symplectic* method.

```
In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= laplace 1
Out[5]= [0.5999999974000001, 0, 0]
In[6]:= method sprk mclachlan_4_4 0.001
Out[6]= method = ARKODE SPRK ARKODE_SPRK_MCLACHLAN_4_4, fixed dt = 0.001
In[7]:= run 12.6 steps 2
Out[7]= t = 12.6 (12600 solver steps, 2 snapshots, |dE/E| = 9.237e-14)
In[8]:= laplace 1
Out[8]= [0.5999999974140128, -0.00000000034051644837163053, 0]
In[9]:= get obj1.position
Out[9]= [0.3964802853115723, 0.06706198328283366, 0]
```

What to notice. `g_constant = 1e-9` makes $GM_{\text{total}} = 1$ so the orbit has period 2π . `LAPLACE 1` divided by `mk` is the *eccentricity vector*: $e = 0.6$ is read directly off `Out[5]`. After ~ 2 orbits the energy is conserved to 9×10^{-14} and **A** has rotated by only 3×10^{-10} — no spurious perihelion precession. Softening was zeroed because any $\varepsilon \neq 0$ slightly breaks the $1/r^2$ law and *would* precess the axis.

9.2 Example 2 — a thrown ball vs. the textbook formula

A projectile has the closed form $x(t) = v_x t$, $y(t) = y_0 + v_y t - \frac{1}{2}gt^2$. We fly a baseball for 6.12 s and subtract the formula *inside the notebook* using bare expressions.

```
In[1]:= new point { mass = 0.145, position = [0, 1, 0], velocity = [30, 30, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= run 6.12 steps 3
Out[3]= t = 6.12 (13 solver steps, 3 snapshots, |dE/E| = 4.740e-15)
In[4]:= get obj0.position.x
```

```

Out[4]= 183.59999999999999
In[5]:= 30 * 6.12
Out[5]= 183.6
In[6]:= obj0.position.x - 30 * 6.12
Out[6]= -0.000000000000008526512829121202
In[7]:= obj0.position.y - (1 + 30 * 6.12 - 9.81 / 2 * 6.12 * 6.12)
Out[7]= -0.00000000000004263256414560601

```

What to notice. Cells 6–7 are *bare expressions* mixing paths and arithmetic (GET itself would refuse the arithmetic). The trajectory matches the parabola to $\sim 10^{-13}$ — and the adaptive Adams integrator needed only **13 internal steps**, because for a polynomial solution its error estimator lets it take huge strides.

9.3 Example 3 — cyclotron motion: expressions as arguments

A charge in a uniform magnetic field circles with period $T = 2\pi m/(|q|B)$. We *compute* T *inside the RUN command*.

```

In[1]:= new sphere { mass = 2, radius = 0.1, charge = -1.5, velocity = [3, 0, 0] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0, 4]
In[3]:= method bdf
Out[3]= method = CVODE BDF
In[4]:= 2 * pi * 2 / (1.5 * 4)
Out[4]= 2.0943951023931953
In[5]:= run 2 * pi * 2 / (1.5 * 4) steps 8
Out[5]= t = 2.0943951023931953 (318 solver steps, 8 snapshots, |dE/E| = 1.444e-8)
In[6]:= get obj0.position
Out[6]= [0.000000000043106769672882073, 0.000000007220835657713453, 0]
In[7]:= norm(obj0.velocity)
Out[7]= 2.9999999783374913

```

What to notice. After one analytic period the particle is back at the origin to 7×10^{-9} , and `norm()` shows the speed unchanged — the magnetic force does no work. The Lorentz force $q\mathbf{v} \times \mathbf{B}$ is velocity-dependent, so the symplectic path is *not allowed* here; BDF is the right tool for fast gyration.

9.4 Example 4 — the tennis-racket theorem (Dzhanibekov effect)

A rigid body spun about its *intermediate* principal axis is unstable: a tiny wobble grows into a dramatic flip, yet energy and angular momentum stay conserved. Half-extents [0.5, 1, 2] give moments [5, 4.25, 1.25]; spinning about y (the middle value) triggers it.

```

In[1]:= new cuboid { mass = 3, half_extents = [0.5, 1, 2], angular_velocity = [0.01, 3, 0.01] }
Out[1]= obj0
In[2]:= get obj0.inertia_tensor
Out[2]= [[5, 0, 0], [0, 4.25, 0], [0, 0, 1.25]]
In[3]:= get obj0.angular_velocity
Out[3]= [0.010000000000000002, 3, 0.010000000000000002]
In[4]:= run 40 steps 4

```

```

Out[4]= t = 40 (2361 solver steps, 4 snapshots, |dE/E| = 5.605e-9)
In[5]:= get obj0.angular_velocity
Out[5]= [1.5428438559953521, 2.9947703802651175, -0.7871804456905063]
In[6]:= run 40 steps 4
Out[6]= t = 80 (2334 solver steps, 4 snapshots, |dE/E| = 5.332e-9)
In[7]:= get obj0.angular_velocity
Out[7]= [0.020666590902422656, 2.9999548562146483, 0.013346831306280834]
In[8]:= get obj0.angular_momentum
Out[8]= [0.05, 12.75, 0.0125]

```

What to notice. At $t = 40$ the wobble has exploded ($\omega_x \approx 1.54$, mid-flip); by $t = 80$ the body has completed the flip and returned to a clean spin. Throughout, the world-frame angular momentum (Out[8]) is exactly the initial $I\omega = [0.05, 12.75, 0.0125]$ — the solver integrates the full quaternion + angular-momentum state with energy error $\sim 5 \times 10^{-9}$.

9.5 Example 5 — three bodies, then surgery with DEL

```

In[1]:= new point { mass = 1,   position = [1, 0, 0],   velocity = [0, 1, 0] }
Out[1]= obj0
In[2]:= new point { mass = 4,   position = [-1, 0, 0],  velocity = [0, -0.25, 0] }
Out[2]= obj1
In[3]:= new point { mass = 256, position = [0, 8, 0],   velocity = [0, 0, 0] }
Out[3]= obj2
In[4]:= set system.g_constant = 0.001
In[5]:= momentum
Out[5]= [0, 0, 0]
In[6]:= com
Out[6]= [-0.011494252873563218, 7.846743295019157, 0]
In[7]:= run 3 steps 3
Out[7]= t = 3 (84 solver steps, 3 snapshots, |dE/E| = 3.141e-10)
In[8]:= list
Out[8]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
obj2: point, mass=256, charge=0, pos=[-0.000017852126656859762, 7.999648688652149, 0]
In[9]:= del 2
Out[9]= deleted obj2; 2 object(s) remain (indices renumbered)
In[10]:= list
Out[10]= obj0: point, mass=1, charge=0, pos=[0.9936356911262184, 3.022315903943394, 0]
obj1: point, mass=4, charge=0, pos=[-0.9972663866755148, -0.7330950497234504, 0]
In[11]:= momentum
Out[11]= [0.0021430470372574067, 0.061528401914275554, 0]

```

What to notice. Cells 1–3 chose velocities so the initial total momentum is exactly zero ($1 \cdot 1 + 4 \cdot (-0.25) = 0$), and it stays zero through the run. Deleting the heavy body removes its momentum: the remainder (Out[11]) is what the surviving pair carries — conservation applies to the system you keep. DEL renumbers later objects.

9.6 Example 6 — magnetic torque (and why ENERGY grows)

magnetic_moment_tensor M couples the world B field to torque $\tau = (RMR^T)B$. Unlike everything else, this coupling is *not derived from a potential*, so total energy is *expected* to change.

```

In[1]:= new sphere { mass = 1, radius = 0.5, magnetic_moment_tensor = [[0.2, 0, 0], [0,
    0.2, 0], [0, 0, 0.2]] }
Out[1]= obj0
In[2]:= set system.b_field = [0, 0.5, 0]
In[3]:= get obj0.angular_momentum
Out[3]= [0, 0, 0]
In[4]:= energy
Out[4]= 0
In[5]:= run 4 steps 4
Out[5]= t = 4 (214 solver steps, 4 snapshots, |dE/E| = 8.000e-1)
In[6]:= get obj0.angular_momentum
Out[6]= [0, 0.400000000000000085, 0]
In[7]:= get obj0.angular_velocity
Out[7]= [0, 4.0000000000000009, 0]
In[8]:= energy
Out[8]= 0.80000000000000035

```

What to notice. Constant torque $\tau = MB = 0.2 \times 0.5 = 0.1$ about y gives $L(t) = 0.1t$: after 4 s, $L = 0.4$ exactly (the ODE is linear, so the solver nails it). Sphere inertia $= 0.4 \cdot 1 \cdot 0.25 = 0.1$, so $\omega = L/I = 4$ and the rotational energy $\frac{1}{2}\omega L = 0.8$ matches Out[8]. The reported $|dE/E| = 8e-1$ is not an error — it honestly reports that the torque pumped energy in.

9.7 Example 7 — fixing a typo in an old cell with %edit

```

In[1]:= new sphere { mass = 20, radius = 0.5 }
Out[1]= obj0
In[2]:= set obj0.velocity = [1, 0, 0]
In[3]:= get obj0.momentum
Out[3]= [20, 0, 0]
In[4]:= %edit 1 new sphere { mass = 2, radius = 0.5 }
In[4]:= new sphere { mass = 2, radius = 0.5 }
Out[4]= obj1
In[5]:= %history
In[1]:= new sphere { mass = 2, radius = 0.5 }
Out[1]= obj0
In[2]:= set obj0.velocity = [1, 0, 0]
In[3]:= get obj0.momentum
Out[3]= [20, 0, 0]
In[4]:= new sphere { mass = 2, radius = 0.5 }
Out[4]= obj1

```

What to notice. `%edit 1 ...` rewrites cell 1's stored input and re-executes it as the next cell — like editing a Jupyter cell and shift-entering again. Because state is cumulative, the re-executed NEW made a *second* object; the fat `obj0` is still there. For a clean rebuild use `%reset + %rerun` or `%load`; when a field tweak suffices, `set obj0.mass = 2`. `%history` shows the *edited* input but the *original* outputs — an audit trail of what actually ran.

9.8 Example 8 — checking the simulator against itself

```

In[1]:= new point { mass = 2, position = [1, 0, 0], velocity = [0, 3, 0] }

```

```

Out[1]= obj0
In[2]:= cross(obj0.position, obj0.momentum)
Out[2]= [0, 0, 6]
In[3]:= angmom
Out[3]= [0, 0, 6]
In[4]:= dot(obj0.position, obj0.velocity)
Out[4]= 0
In[5]:= norm(cross(obj0.position, obj0.momentum)) / (norm(obj0.position) * norm(obj0.
momentum))
Out[5]= 1

```

What to notice. Cell 2 computes $L = r \times p$ by hand and cell 3 confirms **ANGMOM** agrees. Cell 4 proves $r \perp v$; cell 5 computes $|r \times p|/(|r||p|) = \sin \theta = 1$ — the angle between r and p is 90° — inside one nested expression.

9.9 Example 9 — hand-built tensors and a quaternion orientation

```

In[1]:= new cuboid { mass = 1, inertia_tensor = [[2, 0, 0], [0, 3, 0], [0, 0, 4]],
orientation = [0.7071067811865476, 0, 0, 0.7071067811865476] }
Out[1]= obj0
In[2]:= get obj0.inverse_inertia_tensor
Out[2]= [[0.5, 0, 0], [0, 0.3333333333333333, 0], [0, 0, 0.25]]
In[3]:= set obj0.angular_velocity = [0, 2, 0]
In[4]:= get obj0.angular_momentum
Out[4]= [0.00000000000000004440892098500628, 4.0000000000000002, 0]
In[5]:= get obj0.orientation
Out[5]= quat[w=0.7071067811865476, x=0, y=0, z=0.7071067811865476]

```

What to notice. Because `inertia_tensor` appears in the initializer, the analytic inertia is **not** recomputed — your `diag(2,3,4)` is kept, with the inverse maintained automatically. The quaternion $[w, x, y, z] = [\sqrt{1/2}, 0, 0, \sqrt{1/2}]$ is a 90° rotation about z , so setting the *world* angular velocity $[0, 2, 0]$ exercises the full $L = (RIR^T)\omega$ transformation. By hand: rotating `diag(2,3,4)` by 90° about z swaps the x and y moments, so $RIR^T = \text{diag}(3, 2, 4)$, the world- y moment is 2, and $L = [0, 4, 0]$ — exactly `Out[4]` (up to 4×10^{-16} of floating-point dust).

9.10 Example 10 — reproducibility: %save, %reset, %load

```

In[1]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= step 0.5
Out[3]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[4]:= %save session.posim
saved 3 cell(s) to session.posim
In[4]:= %reset
system reset
In[4]:= list
Out[4]= (no objects)
In[5]:= %load session.posim
In[5]:= new sphere { mass = 2, radius = 0.5, velocity = [1, 0, 0] }
Out[5]= obj0

```



```

In[6]:= set system.gravity = [0, -9.81, 0]
In[7]:= step 0.5
Out[7]= t = 0.5 (advanced by 0.5, 12 solver steps)
In[8]:= get obj0.velocity
Out[8]= [1, -4.905000000000001, 0]

```

What to notice. `%save` writes only the successful, non-magic inputs — a clean, replayable script. After `%reset`, `%load` replays it and the state is bit-identical ($v_y = -9.81 \cdot 0.5 = -4.905$). The same file runs headlessly via `posim -script`.

Notice too that **magics do not consume cell numbers**: `%save` and `%reset` print their message and leave the counter alone, which is why the prompt shows `In[4]` three times in a row before `LIST` finally becomes cell 4. Only executed commands become numbered cells — so the `In[n]` numbering always matches what `%history` will replay.

The machine mode speaks the same language over JSON:

```

$ printf '%s\n' '{"op":"exec","code":"new point { mass = 1, velocity = [0, 1, 0] }"}' \
                '{"op":"get","path":"obj0.momentum"}' \
                '{"op":"set","path":"obj0.mass","value":5}' \
                '{"op":"get","path":"obj0.momentum"}' | posim --machine
{"display":"obj0","ok":true,"result":"obj0"}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}
{"display":"","ok":true,"result":null}
{"display":"[0, 1, 0]","ok":true,"result":[0.0,1.0,0.0]}

```

Changing the mass did **not** change the momentum — momentum is the canonical stored state; the *velocity* is what changed. That final subtlety is the union design showing through the wire protocol.

9.11 Example 11 — watching a Kepler orbit live, then running it backward

The same two-body setup as Example 1 — but this time we *watch* it. `SCENE CREATE` opens the scene window in the browser; `SCENE START` sets it in motion; `SCENE REVERSE` runs the movie backward.

```

In[1]:= new point { mass = 1e9, position = [0, 0, 0] }
Out[1]= obj0
In[2]:= new point { mass = 1, position = [0.4, 0, 0], velocity = [0, 2, 0] }
Out[2]= obj1
In[3]:= set system.g_constant = 1e-9
In[4]:= set system.softening = 0
In[5]:= scene create 7878
Out[5]= scene window created: http://127.0.0.1:7878/
        (opened in your browser; if no window appeared, open that address yourself)
        showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[6]:= scene set_time_step 0.005
Out[6]= scene time step dt = 0.005
In[7]:= scene start
Out[7]= scene playback: running
In[8]:= scene pause
Out[8]= scene playback: paused

```

```

In[9]:= scene status
Out[9]= scene: http://127.0.0.1:7878/ (1 window(s) connected)
mode = paused, t = 0.4550000000000003, dt = 0.005, steps = 91, history = 91 frame(s)
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[10]:= scene reverse
Out[10]= scene playback: reversing
In[11]:= scene status
Out[11]= scene: http://127.0.0.1:7878/ (1 window(s) connected)
mode = reversing, t = 0.15500000000000005, dt = 0.005, steps = 91, history = 31 frame(s)
)
entities = 2 (hidden: none)
camera: yaw = -60°, pitch = 55°, dist = 12, target = [0, 0, 0]
In[12]:= scene events
Out[12]= window connected (1 total)
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)

```

What to notice. Between In[7] and In[8] a few real seconds passed while the orbit ran in the window — playback happens on a background thread at ~ 30 frames per second, so **the notebook prompt never blocks**: cell 9’s STATUS shows the *playback copy* has advanced 91 steps to $t \approx 0.455$ while your notebook state still sits untouched at $t = 0$ (that is the “playback copy” design: the window animates its own synchronized copy of your system, so `GET system.time` here would still print 0). After REVERSE (cell 10), the status in cell 11 shows time flowing *backward* ($t \approx 0.155$) and the history buffer draining ($91 \rightarrow 31$ frames): each reversed frame is an exact recorded snapshot, so the rewind retraces the orbit bit-for-bit. Cell 12 drains the asynchronous event queue — the window announced itself when the browser connected. In JupyterLab you would not even need cell 12: events surface on their own as `[scene] ... lines` (§7).

9.12 Example 12 — driving the camera from the notebook

Every gesture the mouse can make in the window has a command twin, so a *script* can compose the exact view you want — useful for repeatable demonstrations and screenshots.

```

In[1]:= new sphere { mass = 3, radius = 0.6, position = [2, 0, 0] }
Out[1]= obj0
In[2]:= new cuboid { mass = 1, half_extents = [0.4, 0.3, 0.2], position = [-2, 0, 1] }
Out[2]= obj1
In[3]:= scene create 7878
Out[3]= scene window created: http://127.0.0.1:7878/
(opened in your browser; if no window appeared, open that address yourself)
showing 2 entities; SCENE START begins the evolution -- HELP lists all scene commands
In[4]:= scene translate 2 0
Out[4]= camera target = [2, 0, 0]
In[5]:= scene rotate 30 -10
Out[5]= camera yaw = -30°, pitch = 45°
In[6]:= scene zoom in
Out[6]= camera distance = 9.6
In[7]:= scene zoom 2
Out[7]= camera distance = 4.8
In[8]:= scene zoom out
Out[8]= camera distance = 5.999999999999999

```

```

In[9]:= scene hide 0
Out[9]= 1 object(s) hidden
In[10]:= scene show all
Out[10]= 0 object(s) hidden
In[11]:= scene redraw
Out[11]= scene redraw queued for every window
In[12]:= scene status
Out[12]= scene: http://127.0.0.1:7878/ (0 window(s) connected)
mode = stopped, t = 0, dt = 0.01, steps = 0, history = 0 frame(s)
entities = 2 (hidden: none)
camera: yaw = -30°, pitch = 45°, dist = 5.999999999999999, target = [2, 0, 0]
In[13]:= scene close
Out[13]= scene closed (http://127.0.0.1:7878/)

```

What to notice. The camera is an **orbit camera**: it circles a *look-at point* at a *distance*, aimed by *yaw* (compass direction) and *pitch* (elevation). TRANSLATE 2 0 moves the look-at point onto the sphere (this is what the arrow keys do in the window); ROTATE 30 -10 adds 30° of yaw to the default -60° and -10° of pitch to the default 55° (this is what left-dragging does); the three zooms multiply the distance by 1/1.25, then 1/2, then 1.25 — watch the arithmetic: 12 → 9.6 → 4.8 → 6 (this is the mouse wheel). HIDE 0 blanks the sphere out of every connected window without deleting anything — SHOW ALL brings it back. Because this transcript ran headless, cell 12 reports 0 window(s) connected — the commands work regardless, and any window that connects later receives the composed view. Note also the two-argument form **scene translate 2 0**: the optional *dz* defaulted to 0, and -10 in cell 5 was one negative argument, not a subtraction (scene arguments are *terms* — the second subtlety of §3).

9.13 Example 13 — the box of shapes: every body type in a rigid, infinitely massive box

The finale scene: BOX 4 builds the rigid room (§5.8) and one of every shape goes inside ($R = 1$, $M = 1$) — a **torus** (mass M , inner radius 1, outer radius 2), a **point** (M , the only mover: $v = (100, 200, 100)$), a **sphere** ($2M$, $r = \frac{1}{2}$), an ideal zero-thickness **disk** ($2M/3$, $r = 1$), a **cube** ($5M/3$, side 1) and a **cylinder** ($2M$, $r = \frac{1}{2}$, height $3/2$). An axis-aligned torus of outer radius 2 would *exactly inscribe* the 4-box (its outer equator touching four walls), so the torus is tilted with its axis along $(1, 1, 1)/\sqrt{3}$: its extent per axis is $1.5\sqrt{2/3} + 0.5 \approx 1.7247 < 2$ — clearance 0.2753. The other positions are random, drawn by a documented LCG ($x \leftarrow 1664525x + 1013904223 \bmod 2^{32}$, seed 20260724, sequential rejection at ≥ 0.05 separation) — see `scripts/collisions/11_box_of_shapes.posim`, the script this transcript replays.

```

In[1]:= set system.g_constant = 0
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
        obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
        the inside faces
In[3]:= new torus { mass = 1, inner_radius = 1, outer_radius = 2, orientation =
        [0.888073833977115, -0.325057583671868, 0.325057583671868, 0] }
Out[3]= obj6
In[4]:= new point { mass = 1, position = [1.406590, -0.995859, 0.569601], velocity =
        [100, 200, 100] }
Out[4]= obj7

```

```

In[5]:= new sphere { mass = 2, radius = 1/2, position = [1.424704, 1.367496, -0.493612]
}
Out[5]= obj8
In[6]:= new disk { mass = 2/3, radius = 1, position = [-0.677386, -1.041493,
-1.091679], orientation = [0.900447102352677, 0.307567078752479,
-0.307567078752479, 0] }
Out[6]= obj9
In[7]:= new cuboid { mass = 5/3, half_extents = [0.5, 0.5, 0.5], position = [1.074397,
0.816223, 1.102099] }
Out[7]= obj10
In[8]:= new cylinder { mass = 2, radius = 1/2, height = 3/2, position = [-1.027024,
-1.403890, -0.485619], orientation = [0.968912421710645, 0, 0.247403959254523, 0] }
Out[8]= obj11
In[9]:= list
Out[9]= obj0: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[3, 0, 0] [wall: static,
inverse_mass=0]
obj1: cuboid he=[1, 4, 4], mass=0, charge=0, pos=[-3, 0, 0] [wall: static, inverse_mass
=0]
obj2: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, 3, 0] [wall: static, inverse_mass
=0]
obj3: cuboid he=[4, 1, 4], mass=0, charge=0, pos=[0, -3, 0] [wall: static, inverse_mass
=0]
obj4: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, 3] [wall: static, inverse_mass
=0]
obj5: cuboid he=[4, 4, 1], mass=0, charge=0, pos=[0, 0, -3] [wall: static, inverse_mass
=0]
obj6: torus ring=1.5 tube=0.5, mass=1, charge=0, pos=[0, 0, 0]
obj7: point, mass=1, charge=0, pos=[1.40659, -0.995859, 0.569601]
obj8: sphere r=0.5, mass=2, charge=0, pos=[1.424704, 1.367496, -0.493612]
obj9: disk r=1, mass=0.6666666666666666, charge=0, pos=[-0.677386, -1.041493,
-1.091679]
obj10: cuboid he=[0.5, 0.5, 0.5], mass=1.6666666666666667, charge=0, pos=[1.074397,
0.816223, 1.102099]
obj11: cylinder r=0.5 h=1.5, mass=2, charge=0, pos=[-1.027024, -1.40389, -0.485619]
In[10]:= collide
Out[10]= collisions ON (51 collidable pair(s); 0 impulse(s) so far)
In[11]:= get obj0.inverse_mass
Out[11]= 0
In[12]:= get obj6.inertia_tensor
Out[12]= [[1.28125, 0, 0], [0, 1.28125, 0], [0, 0, 2.4375]]
In[13]:= energy
Out[13]= 30000
In[14]:= momentum
Out[14]= [100, 200, 100]
In[15]:= run 0.1 steps 100
Out[15]= t = 0.1 (2121 solver steps, 100 snapshots, |dE/E| = 2.040e-10, 119 collision(s)
) -- CONTACTS lists them)
In[16]:= energy
Out[16]= 29999.999993880127
In[17]:= momentum
Out[17]= [146.48911126803657, 102.1478121131382, 46.01601291636828]
In[18]:= get system.collisions
Out[18]= 119
In[19]:= get system.box

```

```
Out[19]= 4
```

What to notice. Cell 3 sizes the torus by the order-independent `inner_radius + outer_radius` pair; cell 8 uses `height = 3/2` — the *full* height, so `half_height` reads 0.75. COLLIDE counts **51 collidable pairs**: 12 objects make 66 pairs, minus the 15 wall-wall pairs (two static bodies can never collide). Out[12] is the torus's analytic inertia diag(1.28125, 1.28125, 2.4375) — exactly $I_{xy} = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$, $I_z = m(c^2 + \frac{3}{4}a^2)$ for $c = 1.5$, $a = 0.5$. The energy starts at $E_0 = \frac{1}{2} \cdot 1 \cdot |v|^2 = 30000$ **exactly** and, after **119 collisions** in 0.1s, is conserved to $|dE/E| = 2.040 \times 10^{-10}$ — but momentum went from (100, 200, 100) to (146.49, 102.15, 46.02): **not conserved**, because the infinitely massive walls absorb momentum without moving — the physical signature of infinite mass. Along the way the point particle can *thread the torus hole* and passes through the ideal zero-thickness disk (a measure-zero contact) — the ball-vs-anything collision tier is exact SDF geometry, not an approximation — while every finite-size body bounces off both. The walls end the run bit-identically at rest.

9.14 Example 14 — two dumbbells from a user-defined function

The two-release finale: a user function (§5.9) that builds **named dumbbells** (§5.1), called twice to launch a pair of tumbling dumbbells at each other, off-center and spinning. No walls this time — so *all three* conserved quantities must survive the collisions. The script is `scripts/collisions/12_two_dumbbells.p` this transcript replays it.

```
In[1]:= set system.g_constant = 0
In[2]:= def create_dumbbell(name, m1 = 1, m2 = 1, m_rod = 0.5, r1 = 0.25, r2 = 0.25,
    rod_radius = 0.1, length = 1, position = [0, 0, 0], velocity = [0, 0, 0],
    angular_velocity = [0, 0, 0]) {
    new dumbbell as name { m1 = m1, m2 = m2, m_rod = m_rod, r1 = r1, r2 = r2, rod_radius
        = rod_radius, length = length, position = position, velocity = velocity,
        angular_velocity = angular_velocity }
}
Out[2]= function create_dumbbell(11 parameter(s)) defined -- 1 body line(s)
In[3]:= create_dumbbell("dumbell0", 1, 2, 0.5, 0.25, 0.25, 0.1, 1, [-2, 0.15, 0], [1.5,
    0, 0], [0, 0, 0.6])
Out[3]= obj0 as dumbell0
In[4]:= create_dumbbell("dumbell1", 2, 1, 0.4, 0.3, 0.2, 0.08, 1.2, [2, -0.15, 0],
    [-1.5, 0, 0], [0.4, 0, 0])
Out[4]= obj1 as dumbell1
In[5]:= list
Out[5]= obj0: dumbbell r1=0.25 r2=0.25 rod_r=0.1 len=1, mass=3.5, charge=0, pos=[-2,
    0.15, 0]
obj1: dumbbell r1=0.3 r2=0.2 rod_r=0.08 len=1.2, mass=3.4, charge=0, pos=[2, -0.15, 0]
In[6]:= get dumbell0.m1
Out[6]= 1
In[7]:= get dumbell1.m_rod
Out[7]= 0.39999999999999999
In[8]:= get dumbell0.vx
Out[8]= 1.5
In[9]:= energy
Out[9]= 7.865310611764706
In[10]:= momentum
Out[10]= [0.150000000000000036, 0, 0]
```

```

In[11]:= angmom
Out[11]= [0.4443030588235295, 0, -1.5059999999999998]
In[12]:= run 3 steps 60
Out[12]= t = 3 (3861 solver steps, 60 snapshots, |dE/E| = 9.463e-11, 2 collision(s) --
CONTACTS lists them)
In[13]:= energy
Out[13]= 7.865310611020375
In[14]:= momentum
Out[14]= [0.150000000000000036, 0, 0]
In[15]:= angmom
Out[15]= [0.44430305882373156, -0.000000000000009547918011776346, -1.505999999999855]
In[16]:= get system.collisions
Out[16]= 2

```

What to notice. Cell 2 is one DEF whose single body line does everything: `new dumbbell` as `name { ... }` forwards all eleven parameters, and because `name` arrives as a string ("dumbell10", "dumbell11"), each call registers the name it was given — `Out[3]/Out[4]` show both handles, `LIST` shows the part geometry, and cells 6–8 read the parts and the `.vx` shorthand through the names. (`Out[7]` reads 0.4 back as 0.3999999999999999: the parts are stored as mass *fractions* of the total, so one reconstruction rounding step shows through — 15 correct digits.) Then the physics: two tumbling, asymmetric rigid bodies meet off-center, collide **twice** (cell 16), and every conserved quantity survives the real CVODE event handling. Energy: 7.865310611764706 → 7.865310611020375 — the run banner prints `|dE/E| = 9.463e-11`. Momentum: [0.150000000000000036, 0, 0] **bit-identical**. Angular momentum about the origin — the delicate one, orbital $r \times p$ plus spin, exchanged between the two at every off-center impact: conserved to $\sim 10^{-13}$ per component (`Out[11]` vs. `Out[15]`), because the part-wise exact narrow phase applies the action–reaction impulse pair at one shared contact point. Contrast Example 13: there the infinitely massive walls absorbed momentum; here, with no walls, E , P and L all hold at solver precision.

9.15 Example 15 — a particle in a box, solved inside the language

The special functions are not decoration: with `eigenvalues` you can set up and solve a small quantum problem in the notebook itself.

Take a particle in an infinite square well of width $L = 1$ with $\hbar = m = 1$. Discretise $-\frac{1}{2}\psi'' = E\psi$ on 4 interior points, spacing $h = 0.2$. The second-derivative stencil puts $1/h^2$ on the diagonal and $-1/2h^2$ beside it, so the Hamiltonian is a 4×4 tridiagonal matrix you can type out. The exact answer is $E_n = n^2\pi^2/2$, so $E_1 = \pi^2/2$.

```

In[1]:= let h = 0.2
Out[1]= h set
In[2]:= let k = 1 / (h * h)
Out[2]= k set
In[3]:= eigenvalues([[k, -0.5*k, 0, 0], [-0.5*k, k, -0.5*k, 0], [0, -0.5*k, k, -0.5*k],
[0, 0, -0.5*k, k]])
Out[3]= [4.774575140626312, 17.274575140626325, 32.72542485937371, 45.225424859373675]
In[4]:= pi * pi / 2
Out[4]= 4.934802200544679

```

The ground state comes out at 4.7746 against an exact 4.9348 — **3.2% low**. That is not a bug,

and it is worth understanding rather than tightening a tolerance until it goes away. A 3-point stencil on 4 points is a *coarse* grid, and its eigenvalues have the closed form $k(1 - \cos(n\pi/5))$ with $k = 1/h^2 = 25$ — which reproduces all four numbers above exactly ($25(1 - \cos 36^\circ) = 4.7746$). The discretisation error is real, known, and second order in h : refine the grid and it falls off as h^2 .

Note the row shapes. Each row here has four entries, so it lexes as a *quaternion* rather than a list — the bracket literal is overloaded (see §4.1). It works anyway, because every bracket shape is accepted where a numeric list is wanted. You should not have to think about quaternions to type a matrix.

The rest of the family is there to be checked against its own identities, which is the honest way to use a special-function library you did not write:

```
In[5]:= let t = 0.7
Out[5]= t set
In[6]:= chebyshev_t(5, cos(t)) - cos(5 * t)
Out[6]= -0.00000000000000011102230246251565
In[7]:= sph_j(0, 1.3) - sin(1.3) / 1.3
Out[7]= 0
In[8]:= legendre_p(3, 0.4) - 0.5 * (5 * 0.4 * 0.4 * 0.4 - 3 * 0.4)
Out[8]= 0.00000000000000011102230246251565
In[9]:= bessej_array(4, 2)
Out[9]= [0.22389077914123567, 0.5767248077568734, 0.35283402861563773,
         0.12894324947440208, 0.03399571980756844]
```

In[6] is $T_n(\cos \theta) = \cos n\theta$; In[7] is $j_0(x) = \sin x/x$; In[8] is $P_3(x) = (5x^3 - 3x)/2$. All three land at zero or one part in 10^{16} — a single rounding step. In[9] returns the whole table $J_0 \dots J_4$ in one call, which is what a Bessel-expanded propagator actually needs.

Finally, the thing the library refuses to do:

```
In[10]:= hermite_h(2.5, 1)
Err[10]: hermite_h(): argument 1 must be a whole number (an integer order), got 2.5
```

There is no Hermite polynomial of order 2.5. The alternative — silently computing H_2 — would hand you a plausible number with no indication anything went wrong, and you would carry it through the rest of the calculation.

10 Error message tour

you type	you get
get obj7.mass (no obj7)	no object obj7
get obj0.bogus	unknown object field 'bogus' -- see HELP for the field list
[1,0,0] * [0,1,0]	cannot multiply vec3 by vec3 (for vec3*vec3 use dot()/cross())
set = 3	parse error at column 5: expected a path root ('objN' or 'system'), found '='
run 1 under SPRK with $B \neq 0$	SPRK method requires a separable Hamiltonian: magnetic field B must be zero ... use METHOD ADAMS or BDF
bogusname	unknown name 'bogusname' (define it with LET, pass it as a function parameter, or use 'bogusname.field' for a registered object)
get d.m1 (only ball is registered)	no object named 'd' (registered names: ["ball"]); NEW ... AS <name> creates one; positional paths are objN.field, contactK.field, system.field)
new sphere as d (name taken)	the name 'd' already refers to obj0 -- DEL it or pick another name
new sphere as system	'system' is reserved
show nosuch	no user function 'nosuch' -- FUNCS lists the defined ones

Every error names the column or the exact field/feature at fault, and never aborts the session.